Security Policy

🔒 Security Commitment

SmartGen is committed to maintaining a secure and safe platform for all users. We take security seriously and appreciate the community's help in identifying and reporting vulnerabilities.

📋 Supported Versions

Version	Status	Support Until
1.0.x	Active	2027-04-27
0.9.x	Legacy	2026-10-27

🐛 Reporting Security Vulnerabilities

Do NOT Report Publicly

If you discover a security vulnerability, please **do not** create a public GitHub issue. Instead:

1. **Email us directly**: security@connectwithbayezid.it.com
2. **Include details**:
   - Type of vulnerability
   - Location in code (if known)
   - Potential impact
   - Steps to reproduce
   - Your contact information
3. **Response time**: We aim to respond within 48 hours

What to Include in Report

Plain Text

```
Subject: Security Vulnerability Report - SmartGen

Description:
```

```
[Detailed description of the vulnerability]

Type:
- [ ] XSS (Cross-Site Scripting)
- [ ] CSRF (Cross-Site Request Forgery)
- [ ] SQL Injection
- [ ] Authentication/Authorization
- [ ] Data Exposure
- [ ] Other: ___________

Severity:
- [ ] Critical
- [ ] High
- [ ] Medium
- [ ] Low

Steps to Reproduce:
1. Step one
2. Step two
3. Step three

Impact:
[Describe potential impact]

Proof of Concept:
[Code or screenshots demonstrating the issue]

Your Information:
Name:
Email:
Organization (optional):
```

🛡️ Security Best Practices

For Users

1. **Keep Updated**
   - Always use the latest version
   - Check for security updates regularly
2. **Protect Your Data**
   - Don't share sensitive information in QR codes
   - Use HTTPS for all connections
   - Verify QR code contents before scanning

3. **Browser Security**
   - Use updated browser
   - Enable security features
   - Use HTTPS connections only
4. **Privacy**
   - Review privacy policy
   - Understand data handling
   - Control your information

For Developers

1. **Code Security**

```typescript
// ✅ Good: Validate and sanitize input
const sanitizeInput = (input: string): string => {
  return input.replace(/[<>]/g, '');
};

// ❌ Bad: Don't trust user input
const unsafeHTML = userInput;
```

2. **Dependency Management**

```bash
# Keep dependencies updated
pnpm update

# Check for vulnerabilities
pnpm audit

# Fix vulnerabilities
pnpm audit --fix
```

3. **Environment Variables**

```bash
# ✅ Good: Use .env files
DATABASE_URL=mysql://...
API_KEY=secret_key
```

```
# ❌ Bad: Don't hardcode secrets
const apiKey = "secret_key";
```

4. **Authentication**
   - Use secure session management
   - Implement CSRF protection
   - Validate all inputs
   - Use HTTPS only

🔐 Security Features

Built-in Security

✅ **No Data Storage**

- QR codes generated locally
- No server-side storage
- No tracking or logging

✅ **Privacy First**

- No personal data collection
- No third-party integrations
- No cookies without consent

✅ **HTTPS Only**

- All connections encrypted
- SSL/TLS certificates
- Secure headers

✅ **Input Validation**

- All inputs validated
- XSS protection
- CSRF tokens

✅ **Secure Dependencies**

- Regular updates
- Vulnerability scanning

- Minimal dependencies

🔍 Security Scanning

Automated Checks

We perform regular security scans:

1. **Dependency Scanning**

   ```bash
   pnpm audit
   ```

2. **Code Analysis**

   ```bash
   npm run lint
   ```

3. **Type Checking**

   ```bash
   pnpm check
   ```

4. **Build Verification**

   ```bash
   pnpm build
   ```

Third-Party Tools

- GitHub Security Advisories
- Snyk vulnerability scanning
- OWASP dependency checking

📝 Security Checklist

Before Deployment

- [ ] All dependencies updated
- [ ] Security audit passed
- [ ] No hardcoded secrets
- [ ] HTTPS enabled
- [ ] Security headers configured
- [ ] Input validation implemented
- [ ] Error handling proper
- [ ] Logging configured
- [ ] Backups available
- [ ] Monitoring enabled

Regular Maintenance

- [ ] Weekly: Check for updates
- [ ] Monthly: Run security audit
- [ ] Quarterly: Security review
- [ ] Annually: Penetration testing

🚨 Incident Response

If a Vulnerability is Discovered

1. **Immediate Actions**
   - Assess severity
   - Determine impact
   - Identify affected versions
2. **Communication**
   - Notify affected users
   - Provide workarounds
   - Release security patch
3. **Resolution**
   - Fix the vulnerability

    - Test thoroughly
    - Deploy fix
    - Verify resolution
4. **Post-Incident**
    - Document incident
    - Update security measures
    - Communicate lessons learned

📚 Security Resources

OWASP Top 10

- OWASP Top 10
- OWASP Testing Guide

Security Standards

- NIST Cybersecurity Framework
- CIS Controls

Tools

- OWASP ZAP
- Burp Suite
- npm audit

🤝 Security Collaboration

Bug Bounty

We appreciate security researchers! While we don't currently offer a formal bug bounty program, we:

- Recognize and credit researchers
- Provide public acknowledgment
- May offer rewards for critical vulnerabilities

Responsible Disclosure

We follow responsible disclosure practices:

1. **Report privately** - Don't disclose publicly
2. **Allow time** - Give us time to fix (typically 90 days)
3. **Coordinate** - Work with us on timeline
4. **Verify** - Confirm fix before public disclosure

📞 Contact

Security Issues

- 📧 Email: security@connectwithbayezid.it.com
- 🌐 Website: www.ConnectWithBayezid.it.com
- 💼 LinkedIn: linkedin.com/in/sayadbayezid

General Inquiries

- 📧 Email: contact@connectwithbayezid.it.com
- 🐙 GitHub: github.com/bayzed123

📄 License

This security policy is licensed under the MIT License.

Last Updated: 2026-04-27

Next Review: 2026-07-27

Thank you for helping keep SmartGen secure! 🙏